

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 8	CLOs to be covered: CLO3
Lab Title: Sets, Dictionary	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 8

Lab Goals / Objectives:

- Understand the concept of sets and their properties
- Learn to create, modify, and perform operations on sets
- Understand the concept of dictionaries as key-value pairs
- Learn to create, access, and manipulate dictionaries
- Master set operations: union, intersection, difference
- Understand dictionary methods and iteration techniques
- Apply sets and dictionaries to practical problems

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. Sets in Python

A set is an unordered collection of unique elements. Sets are mutable but can only contain immutable (hashable) elements. Sets are defined using curly braces `{}` or the `set()` constructor.

```
my_set = {1, 2, 3, 4, 5}
```

Key properties of sets:

- Unordered: Elements have no defined order
- Unique: Duplicate elements are automatically removed
- Mutable: Elements can be added and removed
- Elements must be immutable (numbers, strings, tuples)

2. Creating Sets

- Using curly braces: `s = {1, 2, 3}`
- Using `set()` constructor: `s = set([1, 2, 3])`
- Empty set: `s = set()` (Note: `{}` creates an empty dictionary)

3. Set Methods

- `add(item)` - Adds a single item to the set
- `update(iterable)` - Adds multiple items from an iterable
- `remove(item)` - Removes an item (raises error if not found)
- `discard(item)` - Removes an item (no error if not found)
- `pop()` - Removes and returns an arbitrary item
- `clear()` - Removes all items from the set

4. Set Operations

- Union (| or union()): Combines elements from both sets
- Intersection (& or intersection()): Elements common to both sets
- Difference (- or difference()): Elements in first set but not second
- Symmetric Difference (^): Elements in either set but not both
- Subset (<= or issubset()): Checks if all elements are in another set
- Superset (>= or issuperset()): Checks if set contains all elements of another

5. Dictionaries in Python

A dictionary is an unordered collection of key-value pairs. Each key is unique and maps to a value. Dictionaries are defined using curly braces {} with key:value syntax.

```
my_dict = {"name": "Ali", "age": 20, "city": "Lahore"}
```

Key properties of dictionaries:

- Keys must be unique and immutable (strings, numbers, tuples)
- Values can be any data type
- Dictionaries are mutable
- Dictionaries are optimized for fast key lookup

6. Accessing Dictionary Elements

Values are accessed using their keys:

```
student = {"name": "Ali", "age": 20}  
print(student["name"])    # Ali  
print(student.get("age")) # 20
```

7. Dictionary Methods

- `keys()` - Returns all keys in the dictionary
- `values()` - Returns all values in the dictionary
- `items()` - Returns all key-value pairs as tuples
- `get(key, default)` - Returns value for key, or default if not found
- `update(dict)` - Merges another dictionary
- `pop(key)` - Removes and returns the value for the given key
- `popitem()` - Removes and returns the last inserted key-value pair
- `setdefault(key, value)` - Sets a key if it does not exist
- `clear()` - Removes all items

8. Iterating Over Dictionaries

```
student = {"name": "Ali", "age": 20}
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, value)
```

9. Nested Dictionaries

Dictionaries can contain other dictionaries as values, allowing representation of complex data structures:

```
students = {
    "student1": {"name": "Ali", "age": 20},
    "student2": {"name": "Ahmed", "age": 21}
}
```

10. When to Use Sets vs Dictionaries

- Use sets when you need an unordered collection of unique items and need to perform set operations

- Use dictionaries when you need to associate values with keys for fast lookup

Lab Work:

Task 1: Set Update and Union

Write a program that combines two sets using `update()` and `union()`, and prints the results.

Code Summary:

This task shows `update()`, which merges another set's elements into the first set in place, and `union()`, which returns a new combined set without modifying either original set.

Key Code Lines:

```
# Task 1: Set Update and Union
s1 = {"Ali", "Bilal", "Zain", "Ahmad"}
s2 = {45, 47, 28, 49}
s1.update(s2)
print(s1)
print(s2)
z = s1.union(s1)
print(z)
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"
{'Ahmad', 28, 'Zain', 45, 47, 49, 'Bilal', 'Ali'}
{49, 28, 45, 47}
{'Ahmad', 28, 'Zain', 45, 47, 49, 'Bilal', 'Ali'}
Process finished with exit code 0
```

Task 2: Set Intersection

Write a program that finds the common elements between two sets using `intersection()`.

Code Summary:

This task demonstrates `intersection()`, which returns only the elements that are present in both sets being compared.

Key Code Lines:

```
# Task 2: Set Intersection
s1 = {1, 2, 3}
s2 = {"UET", "CS", "CE", 2}
s3 = {7, 5, 9}
c = s1.intersection(s2)
print(c)
```

Output:

A screenshot of a Python IDE window titled 'Run' with tabs for 'main' and a Python icon. The terminal shows the command 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"' and the output '{2}'. Below the command, it says 'Process finished with exit code 0'. The left sidebar contains icons for file operations like up/down arrows, run, save, print, and delete.

Task 3: Dictionary Lookup by Key

Write a program that stores student marks in a dictionary and looks up a mark by student name.

Code Summary:

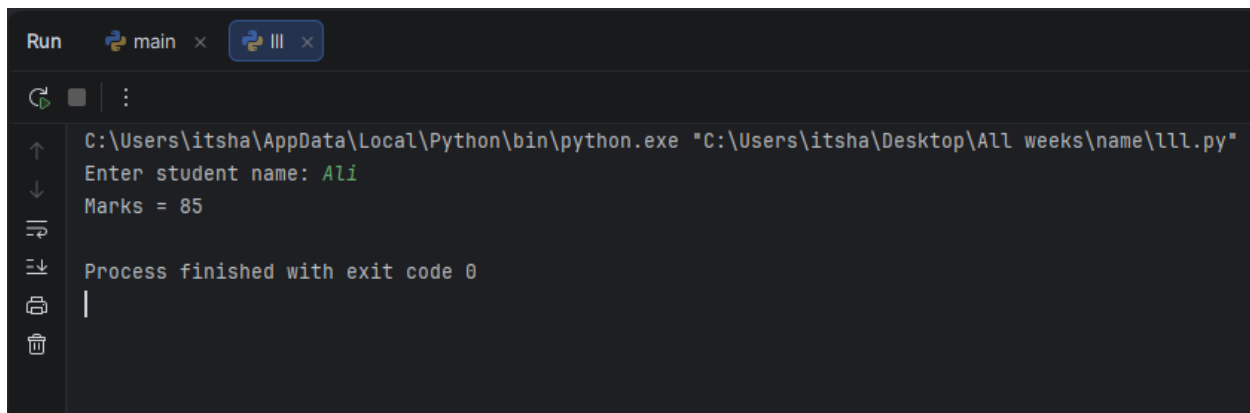
This task demonstrates checking for key existence with the `in` operator before retrieving the associated value from a dictionary.

Key Code Lines:

```
# Task 3: Dictionary Lookup by Key
marks = {
    "Ali": 85,
    "Hamza": 90,
    "Ahmed": 78
}
name = input("Enter student name: ")

if name in marks:
    print("Marks =", marks[name])
else:
    print("Student Not Found")
```

Output:



```
Run main x III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
Enter student name: Ali
Marks = 85
Process finished with exit code 0
```

Task 4: Dictionary keys() and values()

Write a program that creates a dictionary of student details and displays its keys and values separately.

Code Summary:

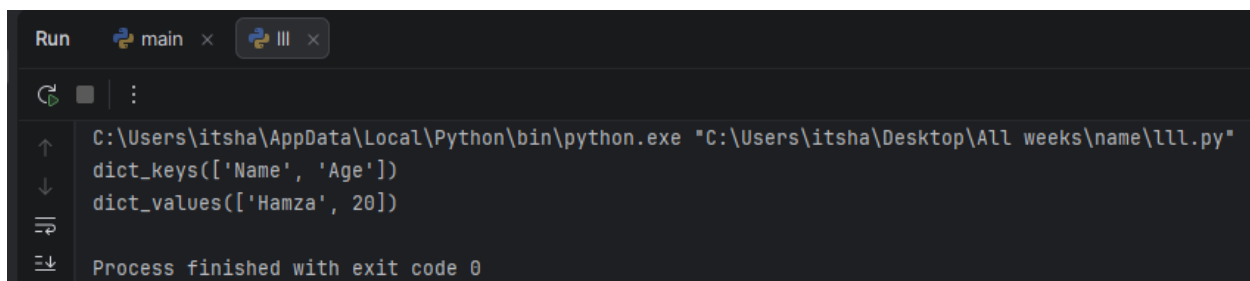
This task demonstrates the keys() and values() dictionary methods, which return view objects of a dictionary's keys and values respectively.

Key Code Lines:

```
# Task 4: Dictionary keys() and values()
student = {"Name": "Hamza", "Age": 20}
print(student.keys())
```

```
student = {"Name": "Hamza", "Age": 20}
print(student.values())
```

Output:



```
Run main x III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
dict_keys(['Name', 'Age'])
dict_values(['Hamza', 20])
Process finished with exit code 0
```

Conclusions:

- Successfully understood and created sets and dictionaries in Python
- Mastered set operations: union, intersection, difference, and symmetric difference
- Learned set methods for adding, removing, and querying elements
- Understood dictionary structure as key-value pairs
- Mastered dictionary access, modification, and iteration techniques